

Lab 5: Memory Request Scheduling

Introduction

Task 1: Getting Your Hands on Ramulator 2

Question 1: Baseline System Configuration

DRAM Architecture & Organization

- Standard: DDR4
- Organization Preset: DDR4_8Gb_x8 (8 Gigabit density per chip, with an 8-bit data interface).
- Channels: 1 channel.
- Ranks: 1 rank per channel.

Timing & Performance

- Timing Preset: DDR4_2400R (Operating at 2400 MT/s).
- Clock Ratio: 3 (The memory system operates at a frequency relative to the simulation base clock).

Memory Controller Logic

- Controller Implementation: Generic
- Scheduling Policy: FRFCFS (First-Ready, First-Come-First-Serve), which prioritizes requests that hit an open row (row-buffer hits) and then older requests.
- Refresh Management: AllBank (Refreshes all banks simultaneously).

System Integration

- Address Mapping: RoBaRaCoCh (Row-Bank-Rank-Column-Channel). This mapping scheme determines how physical addresses are translated into DRAM coordinates, prioritizing row locality.
- Memory System Implementation: GenericDRAM.
- Frontend Connection: It is connected to a SimpleO3 (Out-of-Order) CPU model with a 64KB Last Level Cache (LLC).

Question 2

- The frontend's YAML configuration accepts a list of traces. Each trace in the list corresponds to a CPU core. The `SimpleO3` frontend simulation length is determined by the number of retired instructions, not a fixed number of CPU cycles.
- The simulation terminates once all cores have retired the specified number of instructions.

Question 3

I ran one multicore simulation (2C L-Trace, 2C H-Trace) and two single core simulations (once 1C L-Trace, once 1C H-Trace) for 300K instructions per core per simulation. The number of CPU cycles per core can be gathered from the `cycles_recorded_core_x` statistic printed. The IPC is derived as:

$$\text{IPC} = \text{num_expected_insts} / \text{cycles_recorded_core_X}$$

Trace Type	Single-core IPC	Multi-core IPC	Diff
High Intensity (H)	0.602	0.416	-31%
Low Intensity (L)	3.22	2.174	-32%

Explanation of Results:

The significant drop in IPC (~30%) for both application types in the multi-core scenario is caused by resource contention and inter-application interference in the shared memory system.

1. **Shared Channel Bandwidth:** All four cores share a single DDR4 channel. In the single-core baseline, the application has exclusive access to the channel’s bandwidth. In the multi-core scenario, the high-intensity (“H”) applications compete for this limited bandwidth, saturating the data bus and slowing down their own execution.
2. **Queuing Delays (Latency):** The memory controller’s request queue is shared. The “H” applications generate a large volume of requests that fill the queue. The low-intensity (“L”) applications, which are latency-sensitive, suffer a performance penalty because their occasional requests get stuck behind the heavy traffic of the “H” applications, significantly increasing their average memory access latency.
3. **Bank Conflicts:** With four applications running, there is a higher probability of requests from different cores accessing different rows in the same bank. This leads to row-buffer conflicts.

Question 4

The TraceRecorder plugin is a memory controller plugin designed to log a cycle-accurate trace of all DRAM commands issued by the controller to a file. It creates a per-channel trace file at the path specified in the configuration. Every time the memory controller issues a command, the plugin records a new entry containing the following data:

- Timestamp
- Command Name
- Address Vector

Question 5

1. The Difference Between FCFS and FRFCFS

- **FCFS (First-Come-First-Serve):** Prioritizes requests solely based on when they arrived at the memory controller. If the oldest request is a row-buffer miss, the controller waits for the necessary PRE and ACT commands to complete before issuing any other requests, even if newer requests in the buffer are row-buffer hits.
- **FRFCFS (First-Ready, First-Come-First-Serve):** Adds a “First-Ready” prioritization layer. It first checks if any requests are “ready” (i.e., they are row-buffer hits and satisfy all timing constraints). It prioritizes these ready requests over older requests that are not yet ready. It only falls back to arrival time (FCFS) if multiple requests are ready or if no requests are ready.

2. Impact on System Performance The impact is significant, as seen in the multicore simulation results in the same setup as question 3:

Metric	FRFCFS Policy	FCFS Policy	Performance Impact
Memory System Cycles	270,168	789,751	~2.9x Slower
Core 0 Cycles (L-trace)	134,055	344,892	~2.6x Slower
Core 2 Cycles (H-trace)	720,447	2,097,535	~2.9x Slower

Task 2 & 3: Implementation Details

ATLAS Implementation

My implementation of ATLAS is divided into three components to cleanly separate state management, service tracking, and scheduling logic. It follows the original paper’s mechanisms (Kim et al. 2010) as closely as possible:

1. **The Meta-Controller (AtlasSharedState):** I implemented a global shared structure, `AtlasSharedState`, to act as the “Meta-Controller” described in the ATLAS paper. This structure maintains the global `total_as` (Total Attained Service) for each thread and the resulting `ranking`. It facilitates coordination between memory controllers by aggregating local service stats at the end of each quantum (default 10M cycles). When a quantum expires, it updates the historical AS using the smoothing parameter (set to 0.875) and sorts threads to generate a new rank order, where threads with the least attained service receive the highest priority (lowest rank index).
2. **The Plugin (ATLASPlugin):** The plugin is responsible for tracking the “Attained Service” (AS) during a quantum. I implemented a bank ownership model where a thread “owns” a bank from the moment it issues an ACT command until it issues a PRE command.

- Bank Ownership: A map `bank_ownership` tracks which thread currently has a row open in each bank.
 - AS Calculation: On every cycle, I iterate through all active banks. If a thread owns a bank, its local AS is incremented by 1.
 - Quantum Management: The plugin tracks the cycle count and triggers the `check_in_quantum` routine in the shared state when the quantum limit is reached.
3. **The Scheduler (ATLAS):** The scheduler implements the specific comparison logic to prioritize requests based on the global ranking. The comparison hierarchy is:
- Threshold Check: Requests that have waited longer than the `threshold` (100K cycles) are prioritized to prevent starvation.
 - Rank-Based Prioritization: If neither (or both) requests exceed the threshold, the request from the thread with the *lower* rank index (Least Attained Service) is prioritized.
 - Row-Hit & FCFS: Standard FR-FCFS logic is used as a tie-breaker.

BLISS Implementation

My implementation of BLISS implements the original paper’s mechanism (Subramanian et al. 2016) as closely as possible.

1. **State Management (BLISSState):** A shared state map `bliss_states` stores the blacklist status and request counters for each DRAM channel. This ensures the scheduler and plugin view a consistent system state.
2. **The Plugin (BLISSPlugin):** The plugin monitors the intensity of applications to identify interference-causing threads.
 - Interference Detection: It tracks `consecutive_requests` from the same source ID.
 - Request Counting: A crucial detail in my implementation is strictly adhering to the definition of an “Application Request.” I only increment the consecutive counter when a request is *completed* (i.e., when `req.command == req.final_command`), rather than on every DRAM command (like `ACT` or `PRE`). This prevents false positives where a single read request (generating 3 commands) would otherwise trigger a blacklist threshold of 4.
 - Blacklisting: If a thread exceeds the threshold (set to 4), it is added to the `blacklist`.
 - Clearing: The blacklist is cleared periodically (every 10,000 cycles) to allow applications to recover their priority.
3. **The Scheduler (BLISSScheduler):** The scheduling logic is simplified to a single priority check before the standard memory interactions:
 - Blacklist Check: Requests from non-blacklisted applications are prioritized over blacklisted ones.
 - FR-FCFS: If both applications have the same blacklist status, the scheduler falls back to Row-Hit First, then First-Come-First-Serve.

Task 4: Evaluation & Analysis

1. Instruction Throughput Analysis

The “System Instruction Throughput” chart (fig. 1) reveals the distinct strengths of the scheduling policies:

- **Mixed Workloads (3L-1H, 2L-2H):** ATLAS and BLISS significantly outperform FCFS and slightly outperform FR-FCFS. In the 3L-1H configuration, ATLAS achieves the highest throughput (~11.19 IPC). This confirms that both schedulers successfully identify and deprioritize the high-intensity thread. By allowing the 3 low-intensity threads to be serviced quickly, the overall system throughput increases because these threads are latency-sensitive.
- **Homogeneous High-Intensity Workloads (0L-4H, 0L-8H):** In these saturation scenarios, FR-FCFS and BLISS dominate.
- **ATLAS Performance Collapse:** ATLAS performs poorly (~0.28 IPC for 0L-4H vs ~0.58 for FR-FCFS). This is likely due to the strict ranking combined with a long quantum (10M cycles). In a homogeneous workload, ATLAS picks one “winner” and prioritizes it exclusively for a long duration, effectively serializing the execution and destroying the Bank-Level Parallelism (BLP) that FR-FCFS exploits.
- **BLISS Robustness:** BLISS performs almost identically to FR-FCFS. Since all threads in a 0L-4H workload are “interference-prone,” they all eventually get blacklisted (or fluctuate together). When everyone is blacklisted,

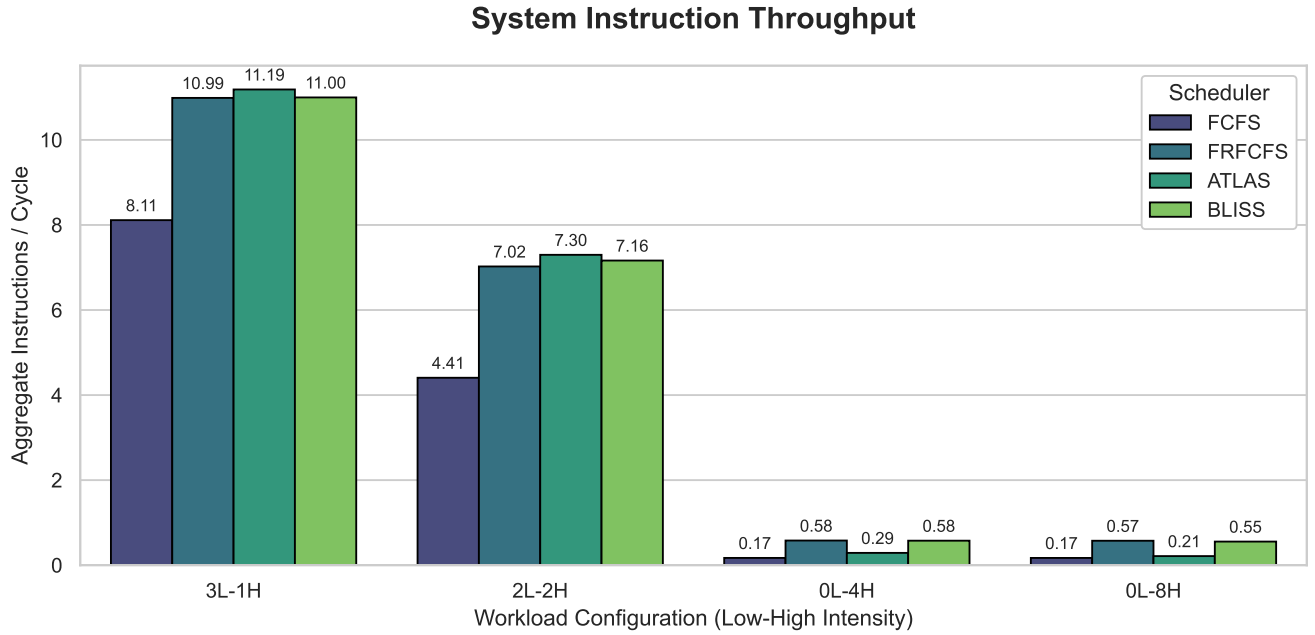


Figure 1: Task 4: **System instruction throughput** for mixed and homogenous workloads on 20M instructions per core. For example 3L-1H has three low memory-intensity traces, one high memory-intensity trace.

the BLISS comparator falls through to its tie-breaker: FR-FCFS. This demonstrates BLISS is more robust than ATLAS in scenarios where ranking is unnecessary.

2. Maximum Slowdown (Unfairness) Analysis

The “Maximum Slowdown” metric plotted in (fig. 2) highlights the fairness trade-offs:

- **The ATLAS Outlier (0L-8H):** The most striking result is the massive unfairness of ATLAS in the 0L-8H case, reaching a slowdown of $>16\times$. This confirms the “serialization” hypothesis: with 8 high-intensity threads and a strict priority system, the lowest-ranked threads are likely starved for nearly the entire 10M cycle quantum.
- **BLISS Stability:** BLISS maintains a much lower maximum slowdown ($\sim 7.3\times$ for 0L-8H), comparable to FR-FCFS ($\sim 7.0\times$). Because BLISS clears its blacklist frequently (every 10k cycles) and does not enforce a strict total order, it prevents the extreme starvation seen in ATLAS.
- **Mixed Workloads:** In the 3L-1H case, all schedulers maintain reasonable fairness (Slowdown < 1.3), but ATLAS and BLISS achieve this while delivering higher throughput, validating their design goals for heterogeneous systems.

Conclusion

- ATLAS provides the highest peak performance for mixed workloads but is fragile in homogeneous, memory-intensive scenarios due to its coarse-grained quantum and strict ranking.
- BLISS offers a more balanced trade-off, delivering high performance in mixed workloads while degrading gracefully to FR-FCFS behavior in saturation scenarios, avoiding the extreme unfairness pitfalls of ATLAS.
- FR-FCFS is an incredibly strong baseline, that offers similar performance to BLISS and ATLAS with much lower complexity.

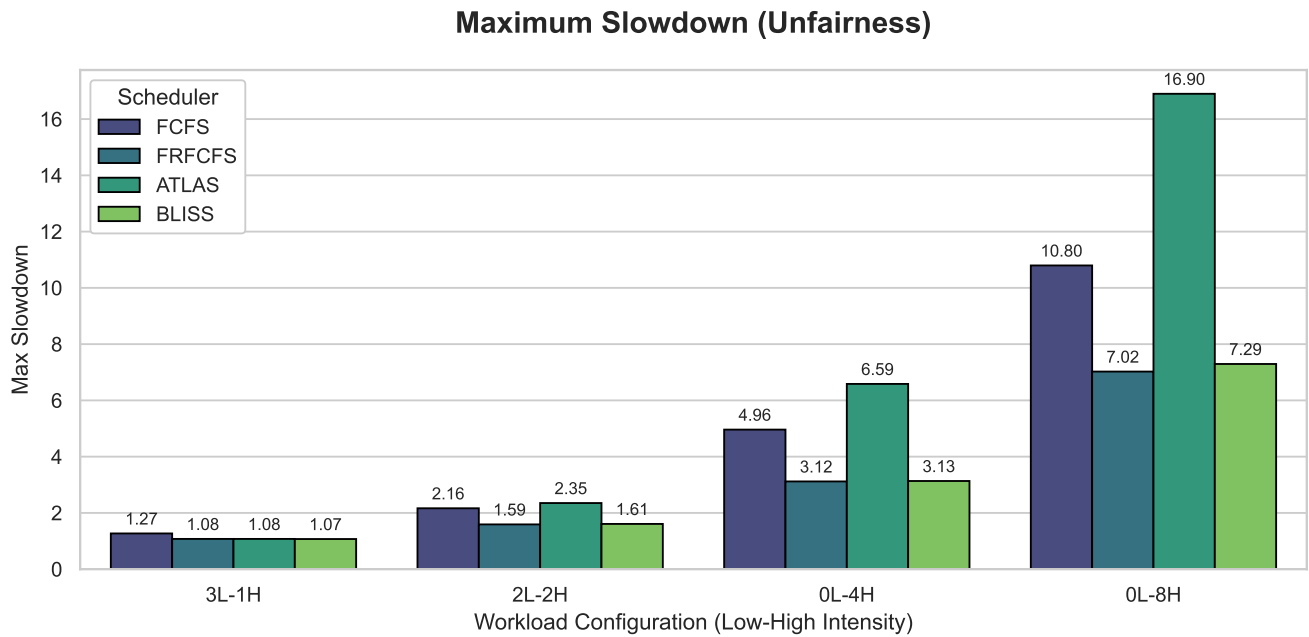


Figure 2: Task 4: **Maximum slowdown** for mixed and homogenous workloads on 20M instructions per core.

Citations

- Kim, Yoongu, Dongsu Han, Onur Mutlu, and Mor Harchol-Balter. 2010. “ATLAS: A Scalable and High-Performance Scheduling Algorithm for Multiple Memory Controllers.” *HPCA - 16 2010 the Sixteenth International Symposium on High-Performance Computer Architecture*, 1–12. <https://doi.org/10.1109/HPCA.2010.5416658>.
- Subramanian, Lavanya, Donghyuk Lee, Vivek Seshadri, Harsha Rastogi, and Onur Mutlu. 2016. “BLISS: Balancing Performance, Fairness and Complexity in Memory Access Scheduling.” *IEEE Transactions on Parallel and Distributed Systems* 27 (10): 3071–87. <https://doi.org/10.1109/TPDS.2016.2526003>.